МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ

ОТЧЕТ
по учебной вычислительной (ознакомительной) практике

Ваткина Тихона Максимовича
студента 1 курса, 4 группы
специальность «Информатика»

Руководитель практики:
старший преподаватель
О.М. Кондратьева

Минск, 2025

Оглавление

Введение....3
Глава 1 Теоретические основы 4
1.1 Общая характеристика курса «Контроль версий с помощью Git» 4
1.2 Общая характеристика курса «Основы Python» 5
1.3 Итоги изучения курсов «Контроль версий с помощью Git», «Основы Python» 8
1.4 Выводы к главе 1 9
Глава 2 Индивидуальный проект «ESCAPER» 11
2.1 Исследование рынка (Market Research)....11
2.2 Краткое описание игры....12
2.3 Этап производства разработки проекта «ESCAPER»....14
2.4 Интерфейс приложения 15
2.5 Архитектура проекта 17
2.6 Реализация бизнес-логики приложения....17
2.7 Выводы к главе 2 22
Заключение 23
Список использованных источников....24

ВВЕДЕНИЕ

Целью учебной вычислительной (ознакомительной) практики является закрепление полученных знаний за первый год обучения посредством выполнения специальных учебных заданий, участия в работе над командным проектом.

Задачи практики:

−Изучение инструментов для совместной разработки проектов и контроля версий.

−Подготовка и настройка программных сред и средств тестирования для прохождения тренинга по выбранному направлению.

−Знакомство с IT компанией, возможностями обучения и повышения квалификации, знакомство с разными направлениями профессиональной подготовки IT специалистов.

−Выполнение стандартных заданий.

−Выполнение заданий повышенного уровня.

−Разработка приложения «ESCAPER».

−Оформление отчета.

Направлением работы было изучение инструментов для совместной разработки проектов и контроля версий на примере Git; изучение высокоуровневого языка программирования Python.

Во время УП 10.07.2025 для студентов была проведена экскурсия в IT компанию «Leverx». На этой встрече мы познакомились с внутренним устройством здания: где расположены переговорные, кабинеты, комнаты отдыха.

LeverX строит решения на базе SAP и современных языков программирования:

−SAP: ABAP, S/4HANA, ECC, UI5/Fiori, SAP Ariba, SuccessFactors.

−Backend: Java, Python, .NET, Node.js.

−Frontend: SAPUI5/Fiori, Angular, React, мобильные SDK.

−Infrastructure: SAP Cloud Platform, HANA.

LeverX – золотой SAP-партнёр и глобальный поставщик стратегических решений:

−Основные клиенты: компании из рейтинга Fortune 500, включая KIA, BMW, Makro, JBS, Girteka, Eurasia Group.

−Партнёрство: SAP, Microsoft Azure, AWS, Google Cloud, Atlassian.

Рассказали о бесплатных курсах:

−Python Programming Course.

−NET Programming Course.

−SAP ABAP Programming Course.

−Node.js Programming Course.

Экскурсия мне понравилась так как я смог ознакомится с их внутренним устройством и узнал истории людей, кто туда устроился.

ГЛАВА 1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

1.1 Общая характеристика курса «Контроль версий с помощью Git»

В данном курсе ознакомился с базовыми возможностями использования Git для совместной разработки проектов. Целью изучения курса было научиться использовать удобный инструмент для контроля версий при программировании приложений. Курс состоит из видеоматериалов с теорией, тестов и практических заданий. Стоит отметить, что информация подавалась в крайне доступном и понятном виде. Были использованы следующие ресурсы:

−Курс «Контроль версий с помощью Git» [1].

−Видео: Git — инструмент для совместной работы с нуля и до регламента в команде [2].

Данные источники предоставляют базовые знания необходимые для освоения возможностей Git.

Курс состоит из следующих модулей:

−Настройка Git.

−Базовые операции.

−Внутреннее устройство git.

−Ветки и merge.

−Stash, tags, remotes.

В модуле «Настройка Git» были пройдены следующие параграфы: VCS concept, Version control types, Why Git, Download, install, configure. Также были изучены основные требования ко всем системам контроля версий, а также их сильные и слабые стороны. Научились устанавливать Git.

В модуле «Базовые операции» были пройдены следующие параграфы: Create a github repo and clone it, Pull from remote. Были изучены способы создания репозиториев, их клонирование, также получение изменений с удалённого репозитория.

В модуле «Внутреннее устройство git» были пройдены следующие параграфы: Git Gui & gitk, Inside .git folder, Undoing changes, Git reset. Были получены сведения о различных инструментах, которые значительно упрощают работу с Git. Также ознакомился с внутренними папками, используемыми Git, и способами хранения информации в Git.

В модуле «Ветки и merge» были пройдены следующие параграфы: Branching and merge, Conflict solving, Rebase, Cherry-pick. Были изучены основные концепты создания и слияния веток, разрешение конфликтов.

В модуле «Stash, tags, remotes» были пройдены следующие параграфы: Tags, Stashing, Remotes, Branching strategies, Extras. Были изучены теги, сохранение изменений без коммита, ихменение удалённого репозитория.

1.2 Общая характеристика курса «Основы Python»

Цель курса «Python» -- обучить студентов основам высокоуровневого языка Python, который можно применять как для быстрой оценки идей, так и для написания продвинутых приложений. Курс состоит из лекций с теорией, тестов и практических заданий. Изложение не всегда было достаточно подробным для понимания, а в задачах иногда приходится использовать знания, которые не были даны на лекциях. Были использованы следующие ресурсы:

−Хендбук по Python от Yandex [3].

−Программирование на языке Python. Основы PyGame [4].

Курс состоит из следующих модулей:

−Коллекции и работа с памятью.

−Функции и их особенности в Python.

−Объектно-ориентированное программирование.

В модуле «Коллекции и работа с памятью» были изучены:

−коллекции, обращение к ним, их встроенные методы, что такое срез, упорядоченные коллекции и индексация;

−множества и словари, их отличия от списков;

−списочные выражения для обработки и создания коллекций, итераторы;

−встроенные возможности по работе с коллекциями (itertools);

−работа с json файлами, потоковый ввод данных, а также выполнение операций чтения и записи с текстовыми файлами.

Мне же в свою очередь наиболее интересными показались списочные выражения, которые позволяют компактно записать требуемый список. Была решена половина всех задач модуля.

Работу большинства изученных в данном модуле объектов можно продемонстрировать на примере задачи:

В этой задаче: список квадратов должен строиться вне зависимости от порядка значений a и b. Если a < b, счёт идёт вперёд. Если a > b, счёт идёт назад. Результат — один список, содержащий квадраты всех чисел от a до b включительно, в соответствующем порядке. Вашему решению даны переменные a и b. Сформируйте список квадратов всех целых чисел от a до b включительно — в прямом или обратном порядке, в зависимости от того, какое значение больше.:

```
[x * x for x in (range(a, b + 1) if a <= b else range(a, b - 1, -1))]
```

В модуле «Функции и их особенности в Python» были изучены:

−синтаксис и примеры создания собственных функций, передача аргументов в функции, возврат значений из функций, локальная и глобальная области видимости;

−передача значений позиционных и именованных аргументов в функции, использование значений по умолчанию, передача функций как аргументов в другие функции, синтаксис лямбда-функций;

−создание рекурсивных функций для реализации декларативного подхода при разработке программ, синтаксис и примеры использования декораторов и генераторов.

Наибольший интерес вызвала работа с функциями в качестве аргументов и возвращаемых значений.

Работу большинства изученных в данном модуле объектов можно продемонстрировать на примере задачи:

Руководство местной кофейни для программистов под названием Java-0x00 решило модернизировать систему заказа кофе.

Для этого им требуется реализовать функцию order, которая принимает список предпочтений посетителя в порядке «убывания желания».

Согласно положению, каждый напиток в кофейне строго определён рецептом:

−Эспрессо готовится из: 1 порции кофейных зерен.

−Капучино готовится из: 1 порции кофейных зерен и 3 порций молока.

−Макиато готовится из: 2 порций кофейных зерен и 1 порции молока.

−Кофе по-венски готовится из: 1 порции кофейных зерен и 2 порций взбитых сливок.

−Латте Макиато готовится из: 1 порции кофейных зерен, 2 порций молока и 1 порции взбитых сливок.

−Кон Панна готовится из: 1 порции кофейных зерен и 1 порции взбитых сливок.

В глобальной переменной in_stock содержится словарь, описывающий ингредиенты в наличии. Ключи словаря: coffee, cream, milk.

Функция должна вернуть:

−название напитка, который будет приготовлен;

−сообщение «К сожалению, не можем предложить Вам напиток», если ни одно из предпочтений не может быть приготовлено.

Если заказ, может быть совершён, количество доступных ингредиентов должно соответствующим образом уменьшиться.:

```
def order(*args):
    temp = in_stock
    GRADES = {
        "Эспрессо": {"coffee": 1},
        "Капучино": {"coffee": 1,
                     "milk": 3},
        "Макиато": {"coffee": 2,
                    "milk": 1},
        "Кофе по-венски": {"coffee": 1,
                           "cream": 2},
        "Латте Макиато": {"coffee": 1,
                          "milk": 2,
                          "cream": 1},
        "Кон Панна": {"coffee": 1,
                      "cream": 1},
```

```
    }
    for grade in args:
        for ingr in GRADES[grade]:
            if GRADES[grade].get(ingr, 0) > in_stock[ingr]:
                break
        else:
            for ingr in GRADES[grade]:
                in_stock[ingr] -= GRADES[grade][ingr]
            return grade
    if in_stock == temp:
        return "К сожалению, не можем предложить Вам напиток"
```

В модуле «Объектно-ориентированное программирование» были изучены:

−классы, их синтаксис, принципы инкапсуляции и полиморфизма.

−наследование классов, множественное наследование, «волшебные» методы Python для реализации стандартных операций с объектами создаваемых классов.

−исключения, иерархия классов исключений, синтаксис обработки исключений, создание собственных исключений, модули.

Наибольший интерес вызвали «волшебные» методы Python и их изменение для получения соответствующих результатов.

Работу большинства изученных в данном модуле объектов можно продемонстрировать на примере задачи:

Рассмотрим объект «Программист», который задаётся именем, должностью и количеством отработанных часов. Каждая должность имеет собственный оклад (заработную плату за час работы). В нашей импровизированной компании существуют 3 должности:

−*Junior* — с окладом 10 тугриков в час;

−*Middle* — с окладом 15 тугриков в час;

−*Senior* — с окладом 20 тугриков в час по умолчанию и +1 тугрик за каждое новое повышение.

Напишите класс Programmer, который инициализируется именем и должностью (отработка у нового работника равна нулю). Класс реализует следующие методы:

−work(time) — отмечает новую отработку в количестве часов time;

−rise() — повышает программиста;

−info() — возвращает строку для бухгалтерии в формате:

<имя> <количество отработанных часов>ч. <накопленная зарплата>тгр.:

```
class Programmer():

    def __init__(self, name, role):
        self.name = name
        self.role = role
        self.time = 0
        self.doxod = 0
        if role == 'Junior':
```

```
            self.den = 10
        elif role == 'Middle':
            self.den = 15
        elif role == 'Senior':
            self.den = 20

    def rise(self):
        if self.role == 'Junior':
            self.role = 'Middle'
            self.den = 15
        elif self.role == 'Middle':
            self.role = 'Senior'
            self.den = 20
        elif self.role == 'Senior':
            self.den += 1

    def work(self, time):
        self.time += time
        self.doxod += time * self.den

    def info(self):
        return f'{self.name} {self.time}ч. {self.doxod}тгр.'
```

1.3 Итоги изучения курсов «Контроль версий с помощью Git», «Основы Python»

При изучении курса «Контроль версий с помощью Git» все модули были проработаны. Результат изучения курса представлен на рисунке 1.1.

Рисунок 1.1 – Общий результат изучения курса по Git

При изучении курса «Основы Python» все модули были проработаны. Результаты изучения курса представлены на рисунках 1.2 – 1.6.

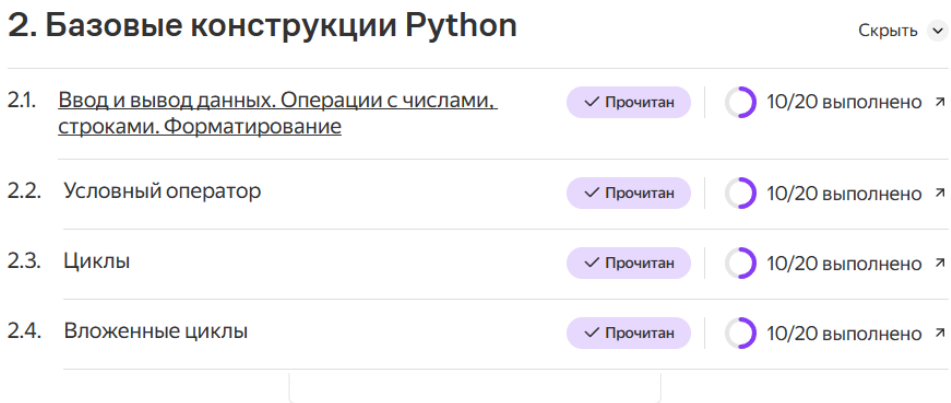


Рисунок 1.2 – Результат изучения 1 раздела

Рисунок 1.3 – Результат изучения 2 раздела

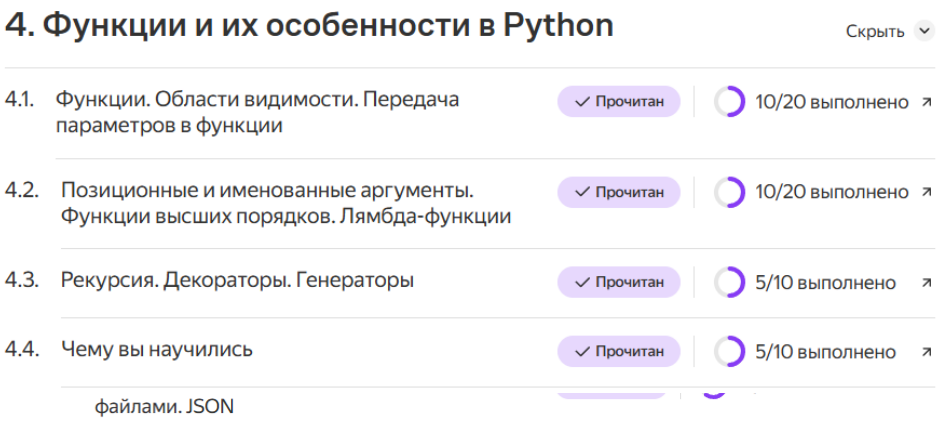


Рисунок 1.4 – Результат изучения 3 раздела

Рисунок 1.5 – Результат изучения 4 раздела

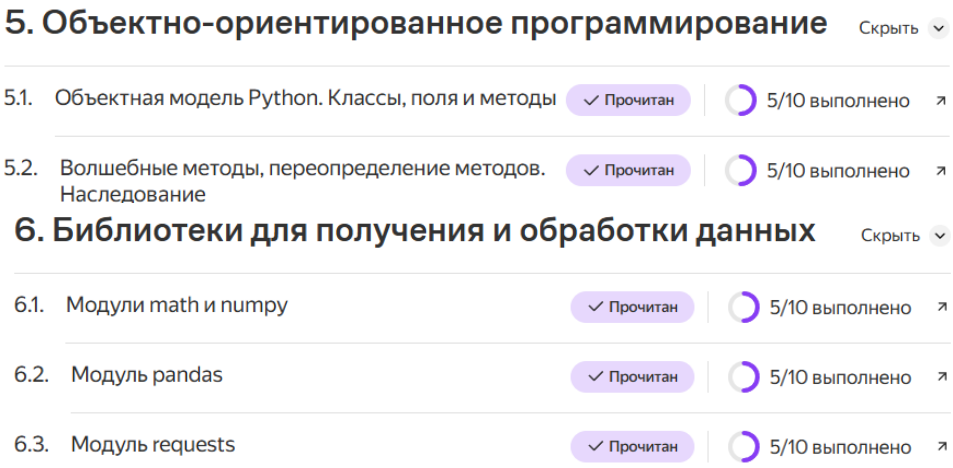


Рисунок 1.6 – Результат изучения 5 раздела

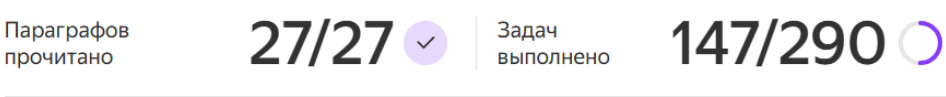


Рисунок 1.7 – Общий результат изучения курса «Основы Python»

1.4 Выводы к главе 1

В процессе учебной практики были изучины инструменты для совместной разработки проектов и контроля версий на примере Git.

Теперь знаем и умеем применять наиболее эффективные подходы к организации групповой деятельности при помощи Git.

В частности, научились создавать репозитории, ветки и управлять этим.

Освоили основы Python и теперь можем использовать его возможности для быстрого написания программ.

Решили около 140 задач, закрепив знания по Python.

Увидели разницу между низкоуровневыми языками программирования (C++) и высокоуровневыми (Python), а также между компилируемыми и интерпретируемыми соответственно.

ГЛАВА 2 ИНДИВИДУАЛЬНЫЙ ПРОЕКТ «ESCAPER»

Вертикальный скролл-шутер в космическом сеттинге. Управление космическим кораблём, который может стрелять. Разные типы врагов, которых надо уничтожать.

USP(Отличительная черта): корабль оснащён особым щитом, который блокирует определённый тип пуль в зависимости от того, какой режим щита выбрал игрок.

Целевая аудитория: игроки, которые любят динамичные игры и не против испытать свои навыки. Возраст: от 8 до 40 лет.

Ожидаемая выгода: оценка со значением “10” за УП.

Творческие цели: игра должна запускаться и не тормозить; должна быть высоко оценена двумя главными критиками и двадцатью шестью меньшего порядка; положительные отзывы (минимум 2).

Технические цели: использование Python и Pygame.

2.1 Исследование рынка (Market Research)

Были изучены эталоны жанра, такие как: Galaga [5], Ikaruga [6], Xevious [7], Battle Garegga [8]. В таблице 2.1 приведена сравнительная характеристика этих игр.

Таблица 2.1 – Сравнение различных вертикальных-скролл шутеров

	Ikaruga	Xevious	Battle Garegga
Геймплей и инновации	Смена полярности (чёрное/белое) с поглощением пуль для заряда атаки	Два типа оружия: зэппер по воздуху и бомбы по земле	Стандартный вертикальный скролл, смена формаций дронов, ранк-
Сложность и кривая обучения	Высокая из-за необходимости точного тайминга смены полярности	Средняя, учится на базовых угрозах и «скрытых» врагах	Очень высокая: динамическое повышение ранга на основе боевых действий
Система оценки и бонусов	Чейны по цвету врагов, абсорбция пуль даёт очки и бонус-атаки	Скрытые «специальные флаги» и башни за бомбы по земле	Медали растущей ценности, бонус за разрушение окружения бомбой

Графика и звук	2D-спрайты на 3D-движке, лаконичный «bullet-hell» саундтрек	2D-графика с палитровыми переходами, минималистичная музыка	Мрачный дизельпанк-стиль, трассирующие пули, агрессивный
Доступность и влияние на жанр	Культовый хит с портами на все платформы после аркадного релиза	Пионер жанра (1983), многочисленные домашние порты	Нишевый шутер для хардкор-энтузиастов

Как видно из таблицы каждый из трёх шутеров отвечает разным запросам: Ikaruga для поклонников головоломок в экшене, Xevious для любителей аркадной классики, Battle Garegga для тех, кто ищет хардкор-опыт и глубокое управление уровнем сложности. Поэтому мы максимально постараемся совместить этот опыт в нашей игре.

2.2 Краткое описание игры

Игрок управляет кораблём, который может стрелять и имеет щит, который блокирует определённый цвет пуль в зависимости от режима щита. Есть враги разных типов, которых нужно уничтожать и от пуль которых нужно уворачиваться. Можно подбирать бонусы, которые изменяют характеристики корабля. Есть меню рейтинга, куда записывается сумма полученных от уничтожения врагов очков.

Жанр, сеттинг, аудитория: Вертикальный скролл-шутер (Shoot ’em up, bullet hell), космос, игроки любящие экшен (от 8 до 40 лет)

Основные механики и геймплей:

−управление перемещением корабля на WASD;

−стрельба на SHIFT;

−временные или постоянные бонусы такие как: неуязвимость, увеличение скорости стрельбы, увеличение мощности пушек, восстановление здоровья щита;

−разные типы врагов, которые различаются по: перемещению, стреляют/ не стреляют, здоровью, направлению атаки; очки набираются от убийства врагов;

−рейтинг;

−пауза;

−3 “жизни” перед смертью;

−музыка и звук.

Условие победы: уничтожение всех боссов (особых врагов), которые появляются при определённом счёте.

Уникальные особенности: щит, который блокирует определённый тип пуль в зависимости от того, какой режим щита выбрал игрок.

Предварительный бизнес-план: монетизация – отсутствует (мы делаем эту игру только ради положительных отзывов и искусства), бюджет – стипендия одного из разработчиков, риски – низкая оценка по УП.

Описание мира игры и основной сюжет:

Действие игры разворачивается в далёком будущем, когда межзвёздные путешествия уже кажутся обычным делом, где-то на окраинах галактики в недружелюбном к людям секторе, наполненном загадочными существами.

Никто из экипажа корабля не может точно сказать: была то пространственная аномалия или чья-то злая шутка – но что они знают наверняка так это то, что и придётся сражаться, чтобы покинуть этот сектор живыми и, может быть, невредимыми.

Персонажи: командир корабля (игрок) и экипаж; инопланетяне.

Описание уровня игры (география, цели, противники): космос; покинуть сектор набрав определённое количество очков; инопланетяне.

Схема уровня: линейное продвижение вперёд.

Интерфейс представлен в виде 4 меню: главное меню, игровой интерфейс, настройки, рейтинг.

Навигация между меню будет осуществляться на кнопки клавиатуры, например ESC и кнопки меню, которые нажимаются мышью.

Стиль игры – Pixelart; спрайты и другие изображения будут взяты из открытого доступа.

Звуковой стиль игры: музыка (динамическая, техно), звуковые эффекты (взрывы и стрельба).

Игра будет написана на языке Python с использованием библиотеки Pygame на Windows 64-bit.

Разработка прототипов основных механик игры: создание прототипа с базовым движением, стрельбой и врагами.

Тестирование различных вариантов геймплея: сбор статистики от волонтёров.

Тестирование прототипов с целевой аудиторией: сбор статистики от волонтёров, например одноклассники и учителя. После сбора обратной связи и анализа результатов проведём внесение изменений в дизайн игры.

Утверждение прототипа, который соответствует видению игры и имеет потенциал для успеха.

Определение необходимого количества людей (программисты, художники, дизайнеры, звукорежиссеры и т.д.): 2 программиста.

Распределение ролей и обязанностей в команде: гибкая адаптация под различные производственные ситуации.

Определение необходимого оборудования и программного обеспечения: компьютер на Windows 64-bit, VS Code, Python, Pygame.

Разработка Плана производства будет осуществляться с 04.07.25 по 10.07.25 включительно находясь в альфа-версии. Затем будет осуществляться доработка до 01.09.25.

Бюджет: стипендия одного из разработчиков.

2.3 Этап производства разработки проекта «ESCAPER»

Цель: Создание всех элементов игры в соответствии с дизайн-документом и планом производства.

В модуле «Разработка графики» будет заимствование текстур в стиле пиксель-арт: корабля, врагов (несколько), бонусов (несколько). Текстуры будут редактироваться при необходимости при помощи Paint.

Будет анимация взрыва корабля посредством перебора картинок взрыва в спрайт листе, который будет взят из открытых источников. Также будет визуальный эффект подбора бонуса неуязвимости: окружность с переменным радиусом.

Программирование игровых механик: движение персонажа, особый щит, бонусы, стрельба, враги (см. дизайн-документ).

Будет простой алгоритм перемещения врагов, их атак.

Звуковые эффекты будут в расширении .ogg и воспроизводится сразу после вызова соответствующих функций, например выстрела и взрыва. Также после запуска игры будет воспроизводится фоновая музыка. Спрайты врагов, бонусов и игрока будут загружаться из заготовленных изображений во время создания объекта.

Программирование пользовательского интерфейса в соответствии с дизайн-документом: главное меню, настройки, рейтинг, пауза, игровое окно.

Оптимизация кода и графики для обеспечения стабильной производительности игры.

План работ по этапу «Программирование»

Даты	**Планируемая деятельность**
02.07.2025	Продолжение углубленного изучения Python
04.07.2025	Окончание углубленного изучения Python и начало изучения Pygame
05.07.2025	Создание динамического прототипа приложения (разработка экранов пользователя)
07.07.2025	Модернизация прототипа: написание реализации базового функционала
08.07.2025	Базовая реализация моделей: врагов, бонусов.
09.07.2025	Добавление спрайтов персонажа
10.07.2025	Интеграция музыки и звуковых эффектам
11.07.2025	Оптимизация и окончательная доработка приложения

В модуле «Дизайн уровней» будет один уровень и он линеен, движение идёт снизу вверх, враги случайно появляются за верхней границей окна сверху и движутся вниз.

Тестирование уровня на предмет удобства навигации, геймплея и производительности будет происходить посредством изменения параметров генерации врагов, размеров окна, генерации предметов.

В модуле «Создание Звука» будут использованы звуки и музыка из открытых источников для звуковых эффектов взрыва и стрельбы, а также BGM (фоновая музыка).

Звуковые эффекты будут в расширении .ogg и воспроизводится сразу после вызова соответствующих функций, например выстрела и взрыва. Также после запуска игры будет воспроизводится фоновая музыка.

В модуле «Интеграция и Тестирование» будет проведена интеграция всех элементов игры (графика, код, звук, уровень).

Будет проведено тестирование игры внутри команды разработчиков с обнаружением и исправлением ошибок.

Будет проведено и справление всех обнаруженных ошибок и багов.

Будет проведена оптимизация производительности игры на основе результатов тестирования.

2.4 Интерфейс приложения

На рисунке 2.1 представлен интерфейс главного меню приложения:

−Play – запуск игрового цикла.

−Leaderboard – переход в меню рейтингов.

−Quit – завершение работы программы.

−Аватар игрока – переход в меню настройки аккаунта.

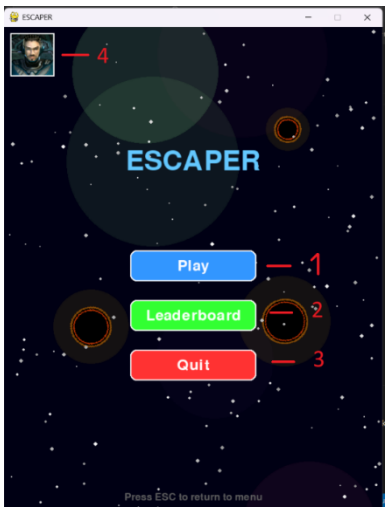
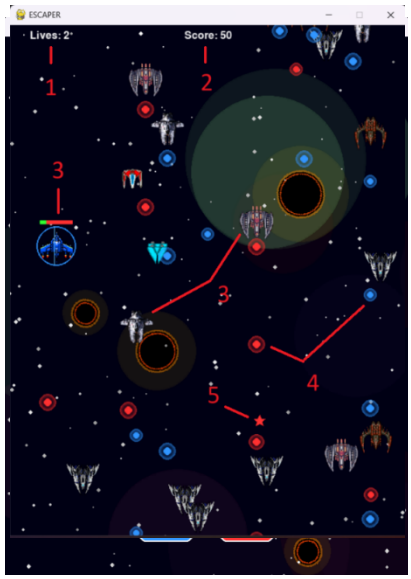


Рисунок 2.1

На рисунке 2.2 представлен интерфейс меню настройки аккаунта:

−Выбор аватара (изображения аккаунта).

−Ввод имени.



−Предпросмотр выбранного корабля (соответствует выбранному персонажу).

−Кнопки OK (выход в главное меню с сохранением изменений) и Cancel без сохранения.

Рисунок 2.2



На рисунке 2.3 представлен интерфейс меню рейтинга 10 лучших игроков, отсортированных по убыванию счёта с отображением: занятого места, аватара, имени и счёта игрока.

Рисунок 2.3

На рисунке 2.4 представлен интерфейс игрового процесса:

−Число жизней.

−Счёт.

−Враги и игрок.

−Пули.

−Бонусы.

Рисунок 2.4

На рисунке 2.5 представлен интерфейс меню паузы с отображением текущего счёта, возможностью выхода в главное меню по нажатию клавиши Escape, возобновление игры на клавишу R

Рисунок 2.5

2.5 Архитектура проекта

Архитектура проекта представлена на следующей UML-диаграмме рисунок 2.7

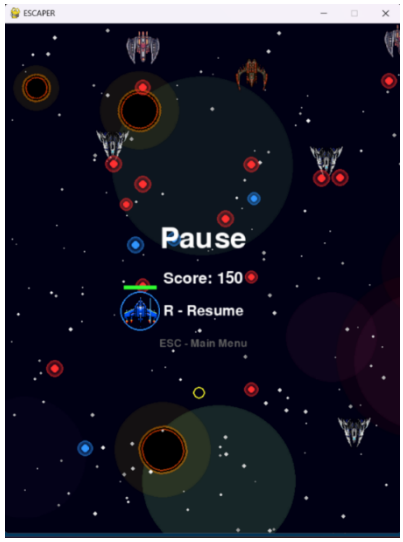


Рисунок 2.7

2.6 Реализация бизнес-логики приложения

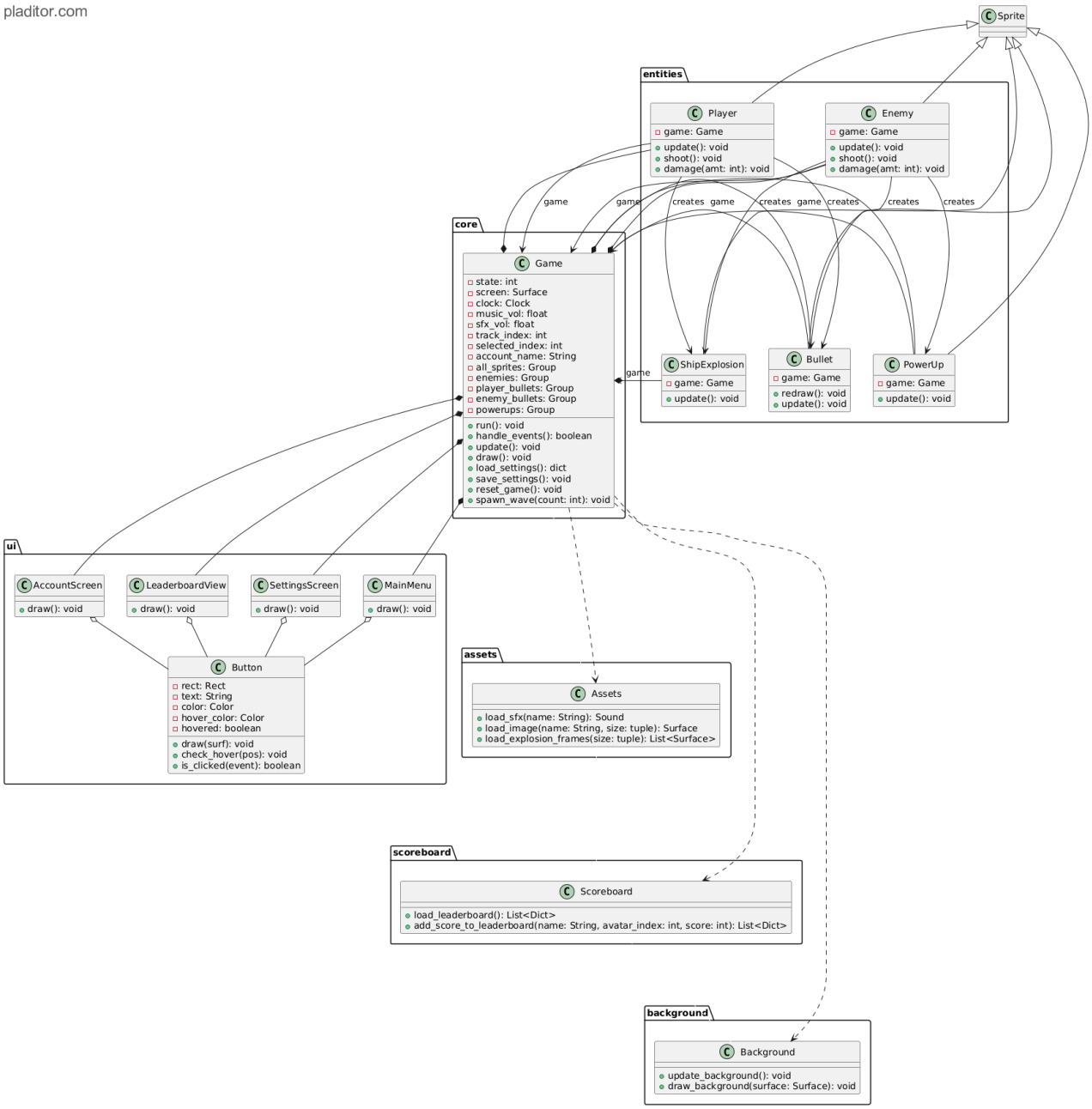
Одним из классов, которые были реализованы мной является класс Player. Класс отвечает за создание и управление главным персонажем, включая его движение и визуальный эффект щита. Класс реализован в файле player.py и наследуется от pygame.sprite.Sprite, что позволяет использовать его в группах спрайтов Pygame для упрощения управления игровыми объектами.

Код класса:

```
import pygame, random, math
from constants          import *
from effects.explosion import ShipExplosion
from entities.bullet   import Bullet

class Player(pygame.sprite.Sprite):
    def __init__(self, ship_index, game):
```

pladitor.com




```
        super().__init__()
        self.game     = game
        self.image    = game.player_ships[ship_index]
        self.rect     =
self.image.get_rect(midbottom=(WIDTH//2, HEIGHT-20))
        self.speed    = 5
        self.health   = 100
        self.lives    = 3
        self.score    = 0
        self.dying    = False

        self.shield_color           = random.choice([RED,
BLUE])
        self.base_delay             = 300
        self.shoot_delay            = 300
```

```
        self.last_shot              = 0
        self.fire_rate_end          = 0
        self.power_level            = 1
        self.power_end              = 0
        self.invulnerable           = False
        self.inv_end                = 0
        self.shield_animation       = 0.0
        self.shield_animation_speed = 0.01
        self.score_saved            = False

    def update(self):
        if self.dying:
            return

        now  = pygame.time.get_ticks()
        keys = pygame.key.get_pressed()

        dx = (keys[pygame.K_d] - keys[pygame.K_a]) *
self.speed
        dy = (keys[pygame.K_s] - keys[pygame.K_w]) *
self.speed
        self.rect.x = max(0, min(WIDTH-self.rect.width,
self.rect.x+dx))
        self.rect.y = max(0, min(HEIGHT-self.rect.height,
self.rect.y+dy))

        if (keys[pygame.K_LSHIFT] or keys[pygame.K_RSHIFT])
and now - self.last_shot >= self.shoot_delay:
            self.last_shot = now
            self.shoot()

        if self.fire_rate_end and now > self.fire_rate_end:
            self.shoot_delay   = self.base_delay
            self.fire_rate_end = 0
        if self.power_end and now > self.power_end:
            self.power_level = 1
            self.power_end   = 0
        if self.invulnerable and now > self.inv_end:
            self.invulnerable = False

        self.shield_animation += self.shield_animation_speed
        if self.shield_animation > 1.0:
            self.shield_animation = 0.0

    def shoot(self):
        patterns = {1:[0], 2:[-10,10], 3:[-15,0,15]}
        for ox in patterns[self.power_level]:
```

```
            b = Bullet(self.rect.centerx + ox, self.rect.top,
                       vy=-10, color=self.shield_color)
            self.game.all_sprites.add(b)
            self.game.player_bullets.add(b)

    def damage(self, amt):
        if self.invulnerable or self.dying:
            return

        self.health -= amt
        if self.health <= 0:
            self.lives -= 1
            if self.lives > 0:
                self.invulnerable = True
                self.inv_end      = pygame.time.get_ticks() +
3000
                self.health       = 100
            else:
                self.dying = True
                exp = ShipExplosion(self.rect.center)
                self.game.all_sprites.add(exp)
                pygame.time.set_timer(pygame.USEREVENT + 1,
500)

    def kill(self):
        pygame.time.set_timer(pygame.USEREVENT + 1, 0)
        super().kill()
```

Инициализация (__init__) – принимает индекс корабля (ship_index) и ссылку на объект игры (game)

Атрибуты:

−game – Ссылка на объект Game для доступа к группам спрайтов и другим данным;

−image – Изображение корабля, выбранное из списка game.player_ships;

−rect – Прямоугольник, определяющий позицию игрока (в центре нижней части экрана);

−speed – Скорость движения (5 пикселей за кадр);

−health – Начальное здоровье (100);

−lives – Начальное количество жизней (3);

−score – Очки игрока (начально 0);

−dying – Флаг состояния смерти;

−shield_color – Случайный цвет щита (красный или синий);

−base_delay, shoot_delay, last_shot – Параметры стрельбый;

−fire_rate_end, power_level, power_end – Параметры бонусов;

−invulnerable, inv_end – Параметры для управления неуязвимостью;

−shield_animation, shield_animation_speed – Анимация щита;

−score_saved – Флаг сохранения очков.

Обновление (update) – делится на:

Движение:

−Обрабатывает ввод с клавиатуры (A, D, W, S) – вычисления смещения;

−Ограничивает движение игрока границами экрана.

Стрельба:

−Проверяет нажатие клавиш Shift и время с последнего выстрела;

−Если условия выполнены, вызывает метод shoot.

Управление бонусами:

−Проверяет истечение времени действия бонусов скорострельности (fire_rate_end), мощности (power_end) и неуязвимости (inv_end);

−Сбрасывает параметры при истечении времени.

Анимация щита:

−Обновляет значение shield_animation для создания визуального эффекта;

−Сбрасывает анимацию при достижении значения 1.0.

Стрельба (shoot) – создает пули в зависимости от уровня мощности (power_level):

−Уровень 1 – Одна пуля в центре.

−Уровень 2 – Две пули с горизонтальным смещением (-10, +10 пикселей).

−Уровень 3 – Три пули с большим смещением (-15, 0, +15 пикселей).

−Каждая пуля создается с вертикальной скоростью -10 и цветом щита игрока.

−Пули добавляются в группы all_sprites и player_bullets.

−Проигрывается звуковой эффект выстрела.

Получение урона (damage):

−Если игрок неуязвим или находится в состоянии смерти, урон игнорируется.

−Уменьшает здоровье на заданную величину (amt).

−Если здоровье достигает нуля:

−Уменьшает количество жизней.

−Если жизни остаются, делает игрока неуязвимым на 3 секунды и восстанавливает здоровье.

−Если жизни закончились, устанавливает флаг dying, создает эффект взрыва (ShipExplosion) и запускает таймер**.**

Уничтожение (kill):

−Отменяет таймер смерти и вызывает метод kill базового класса для удаления игрока из групп спрайтов.

Значение для бизнес-логики:

−Обеспечивает управление: Игрок может перемещаться и стрелять, что составляет основу игрового процесса.

−Поддерживает прогресс: Накопление очков и взаимодействие с бонусами мотивируют игрока продолжать игру.

−Управляет сложностью: Механизмы урона и неуязвимости балансируют сложность, предотвращая слишком быстрое поражение.

−Улучшает визуальный опыт: Анимация щита делает игрока более заметным и добавляет эстетическую ценность.

При разработке класса мне больше всего было интересно реализовывать управление, а также продумывать его взаимодействие с другими сущностями (PowerUp и Enemy)

2.7 Выводы к главе 2

Были выполнены этапы Предпроизводство и Производство игрового проекта «ESCAPER», которые позволили определить основные механики игры, шаги разработки, визуальный стиль и целевую аудиторию проекта, а также продумать будущую архитектуру и определиться с основными требованиями. Также был получен опыт в написании и продумывании всех этапов разработки игры с нуля, что позволит в дальнейшем, не боятся разрабатывать большие проекты, так как будет понятно, с чего начинать и как действовать.

Посредством написания проекта были закреплены и применены знания в языке программирования Python, а именно в разработке классов, написании функций и их взаимодействиях.

Также были изучены основные возможности библиотеки Pygame необходимые для написания игры, в частности для разработки основного цикла игры и обработки событий.

При написании проекта также были закреплены знания и умения в совместной разработке проектов при помощи Git, в частности создание репозитория, коммитов, веток и их слияние.

Как результат всех действий была написана игра «ESCAPER», которая предоставляет достаточный набор функционала и геймплея необходимого для того, чтобы зваться игрой и увлекать игрока.

ЗАКЛЮЧЕНИЕ

В процессе учебной вычислительной (ознакомительной) практики была посещена IT компания «Leverx», получив тем самым сведения о её внутреннем устройстве и возможностях для нашей будущей карьеры.

Были изучены инструменты для совместной разработки проектов и контроля версий на примере Git. Изучение проводилось через курс «Контроль версий с помощью Git», который был полностью изучен и закреплён посредством выполнения всех тестовых заданий курса. Также все полученные знания были использованы при разработке игрового проекта.

Полностью были пройдены все 27 параграфов курса «Основы Python», а также решены 147 задач, тем самым закрепив знания по Python. Также все полученные знания были использованы при разработке игрового проекта.

Были продуманы и описаны этапы Предпроизводство и Производство игрового проекта, а также отредактированы в соответствии с замечаниями.

Сданы вовремя и заполнены все отчёты о деятельности во время учебной практики. Заполнен дневник. Написан данный отчёт.

Также были изучены основные возможности библиотеки Pygame необходимые для написания игры, в частности для разработки основного цикла игры и обработки событий.

Была разработана игра «ESCAPER», которая предоставляет достаточный набор функционала и геймплея необходимого для того, чтобы зваться игрой и увлекать игрока. Лично мной реализованы классы: Player, PowerUp, MainMenu, LeaderboardView, Button.

В дальнейшем планируется доработка проекта в соответствии с новыми требованиями, а так же с запланированным ранее функционалом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Контроль версий с помощью Git [Электронный ресурс]. – Режим доступа: https://learn.epam.com/detailsPage?id=601f195a-d408-4439-a16d-0630ed2a412e – Дата обращения: 12.07.2025.
2. Git — инструмент для совместной работы с нуля и до регламента в команде [Электронный ресурс: видеоролик]. – Режим доступа: https://education.yandex.ru/knowledge/git-instrument-dlya-sovmestnoj-raboty-s-nulya-i-do-reglamenta-v-komande – Дата обращения: 12.07.2025.
3. Хендбук по Python от Yandex [Электронный ресурс]. – Режим доступа: https://education.yandex.ru/handbook/python – Дата обращения: 12.07.2025.
4. Программирование на языке Python. Основы PyGame [Электронный ресурс]. – Режим доступа: https://python-course.readthedocs.io/ru – Дата обращения: 12.07.2025.
5. Galaga [Электронный ресурс]. – Википедия: свободная энциклопедия. – Режим доступа: https://ru.wikipedia.org/wiki/Galaga – Дата обращения: 12.07.2025.
6. Ikaruga [Электронный ресурс]. – Википедия: свободная энциклопедия. – Режим доступа: https://ru.wikipedia.org/wiki/Ikaruga – Дата обращения: 12.07.2025.
7. Xevious [Электронный ресурс]. – Википедия: свободная энциклопедия. – Режим доступа: https://ru.wikipedia.org/wiki/Xevious – Дата обращения: 12.07.2025.
8. Battle Garegga [Электронный ресурс]. – Википедия: свободная энциклопедия. – Режим доступа: https://en.wikipedia.org/wiki/Battle_Garegga – Дата обращения: 12.07.2025.
9. Репозиторий игры Python-Game [Электронный ресурс]. – Режим доступа: https://github.com/Spusik322/Python-Game – Дата обращения: 10.09.2025.